МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
А.С. Кот
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL
Условие: Напишите Python-скрипт, который анализирует самых активных контрибьюторов в указанном GitHub-репозитории за определённый период.

1. Запросить у пользователя:
    - Репозиторий (owner/repo, например, tensorflow/tensorflow).
    - Период анализа (за неделю, за месяц, за год).
    - (Опционально) Минимальное количество коммитов для попадания в рейтинг.Использует GitHub API для получения самых быстрорастущих репозиториев по звёздам (stars).
2. Использовать GitHub API для получения списка контрибьюторов и их активности:
    - Количество коммитов.
    - Количество добавленных и удалённых строк кода.
    - Количество открытых и закрытых pull requests.
    - Количество открытых и закрытых issues.
    - Количество комментариев к PR и issues.
3. Определить ТОП-5 самых активных контрибьюторов.
4. Построить график активности (matplotlib, seaborn).

```
#!/usr/bin/env python3
"""
GitHub Contributor Activity Analyzer
Анализирует активность контрибьюторов в указанном репозитории за определенный период
"""

import requests
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from datetime import datetime, timedelta
from collections import defaultdict
from typing import Dict, List, Tuple, Optional
import numpy as np
from concurrent.futures import ThreadPoolExecutor, as_completed
import time
import sys

# Настройка русских шрифтов для matplotlib
plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial', 'sans-serif']
plt.rcParams['axes.unicode_minus'] = False

class GitHubContributorAnalyzer:
    """Класс для анализа активности контрибьюторов GitHub"""

    def __init__(self, repo: str, token: Optional[str] = None):
        """
        Инициализация анализатора

        Args:
            repo: репозиторий в формате owner/repo
            token: GitHub токен для увеличения лимита запросов (опционально)
```

```
        """
        self.repo = repo
        self.base_url = "https://api.github.com"
        self.headers = {
            "Accept": "application/vnd.github.v3+json",
            "User-Agent": "GitHub-Contributor-Analyzer/1.0"
        }

        if token:
            self.headers["Authorization"] = f"token {token}"
            print("✓ Используется GitHub токен для аутентификации")

        self.owner, self.repo_name = repo.split('/')

        # Периоды в днях
        self.periods = {
            'week': 7,
            'month': 30,
            'year': 365
        }

        # Результаты анализа
        self.contributors = defaultdict(lambda: {
            'commits': 0,
            'additions': 0,
            'deletions': 0,
            'prs_opened': 0,
            'prs_closed': 0,
            'issues_opened': 0,
            'issues_closed': 0,
            'comments': 0,
            'pr_comments': 0,
            'issue_comments': 0,
            'login': '',
            'name': '',
            'avatar': '',
            'activity_score': 0
        })

    def _make_request(self, url: str, params: Dict = None) -> List[Dict]:
        """
        Выполнение запроса к GitHub API с обработкой пагинации
        """
        results = []
        page = 1

        while True:
            if params is None:
                params = {}
            params['page'] = page
            params['per_page'] = 100

            try:
                response = requests.get(url, headers=self.headers, params=params,
timeout=10)
                response.raise_for_status()

                # Проверка лимитов API
                remaining = int(response.headers.get('X-RateLimit-Remaining', 0))
                if remaining < 10:
                    reset_time = int(response.headers.get('X-RateLimit-Reset', 0))
                    wait_time = max(0, reset_time - time.time())
                    if wait_time > 0:
                        print(f"    ⚠ Предупреждение: осталось {remaining} запросов.
Ожидание {wait_time:.0f} секунд...")
                        time.sleep(wait_time)

                data = response.json()
                if not data:
```

```
                    break

                if isinstance(data, list):
                    results.extend(data)
                else:
                    results.append(data)

                # Проверка на наличие следующей страницы
                link_header = response.headers.get('Link', '')
                if 'rel="next"' not in link_header:
                    break

                page += 1

            except requests.exceptions.RequestException as e:
                print(f"  ✗ Ошибка при запросе {url}: {e}")
                break

        return results

    def get_contributors_stats(self, since_date: datetime, min_commits: int = 0) -> Dict:
        """
        Получение полной статистики по контрибьюторам
        """
        print(f"\n📊 Анализ контрибьюторов репозитория {self.repo}")
        print(f"📅 Период анализа: с {since_date.strftime('%Y-%m-%d')} по
{datetime.now().strftime('%Y-%m-%d')}")
        print(f"🔍 Минимальное количество коммитов: {min_commits}\n")

        # Получение списка контрибьюторов
        print("1. Получение списка контрибьюторов...")
        contributors_url = f"{self.base_url}/repos/{self.repo}/contributors"
        contributors = self._make_request(contributors_url)

        # Фильтруем контрибьюторов с достаточным количеством коммитов
        contributors = [c for c in contributors if c['contributions'] >= min_commits]

        print(f"   Найдено {len(contributors)} контрибьюторов (после фильтрации)\n")

        # Собираем данные по каждому контрибьютору
        print("2. Сбор детальной статистики...")

        with ThreadPoolExecutor(max_workers=5) as executor:
            futures = []
            for contributor in contributors:
                login = contributor['login']
                self.contributors[login]['login'] = login
                self.contributors[login]['avatar'] = contributor.get('avatar_url', '')

                # Запускаем задачи параллельно
                futures.append(executor.submit(self._get_commits_stats,           login,
since_date))
                futures.append(executor.submit(self._get_prs_stats, login, since_date))
                futures.append(executor.submit(self._get_issues_stats,            login,
since_date))
                futures.append(executor.submit(self._get_comments_stats,          login,
since_date))

            # Ожидаем завершения всех задач
            for i, future in enumerate(as_completed(futures), 1):
                if i % 20 == 0:
                    print(f"   Обработано {i}/{len(futures)} задач...")

        print("   Сбор данных завершен\n")

        # Рассчитываем общий балл активности
        for login, stats in self.contributors.items():
            stats['activity_score'] = (
                stats['commits'] * 5 +
```

```
                stats['prs_opened'] * 3 +
                stats['issues_opened'] * 2 +
                stats['comments'] * 1 +
                (stats['additions'] + stats['deletions']) / 100  # Нормируем изменения
кода
            )

        # Фильтруем по минимальному количеству коммитов
        filtered = {
            login: stats for login, stats in self.contributors.items()
            if stats['commits'] >= min_commits
        }

        return filtered

    def _get_commits_stats(self, contributor: str, since_date: datetime) -> None:
        """Получение статистики коммитов"""
        commits_url = f"{self.base_url}/repos/{self.repo}/commits"
        params = {
            'author': contributor,
            'since': since_date.isoformat(),
            'until': datetime.now().isoformat()
        }

        commits = self._make_request(commits_url, params)
        self.contributors[contributor]['commits'] = len(commits)

        # Получение детальной статистики изменений для коммитов
        # Ограничиваем последними 30 коммитами для производительности
        for commit in commits[:30]:
            try:
                # Получаем детали коммита
                commit_url = commit['url']
                response = requests.get(commit_url, headers=self.headers, timeout=5)
                if response.status_code == 200:
                    commit_data = response.json()
                    if 'stats' in commit_data:
                        self.contributors[contributor]['additions']                    +=
commit_data['stats'].get('additions', 0)
                        self.contributors[contributor]['deletions']                    +=
commit_data['stats'].get('deletions', 0)
            except Exception:
                continue

    def _get_prs_stats(self, contributor: str, since_date: datetime) -> None:
        """Получение статистики Pull Requests"""
        # Открытые PR
        prs_url = f"{self.base_url}/repos/{self.repo}/pulls"
        params = {
            'state': 'all',
            'since': since_date.isoformat()
        }

        all_prs = self._make_request(prs_url, params)

        for pr in all_prs:
            if pr['user']['login'] == contributor:
                self.contributors[contributor]['prs_opened'] += 1
                if pr['state'] == 'closed':
                    self.contributors[contributor]['prs_closed'] += 1

    def _get_issues_stats(self, contributor: str, since_date: datetime) -> None:
        """Получение статистики Issues"""
        issues_url = f"{self.base_url}/repos/{self.repo}/issues"
        params = {
            'state': 'all',
            'since': since_date.isoformat(),
            'filter': 'all'
        }
```

```
        all_issues = self._make_request(issues_url, params)

        for issue in all_issues:
            # Игнорируем PR, так как они уже учтены
            if 'pull_request' in issue:
                continue

            if issue['user']['login'] == contributor:
                self.contributors[contributor]['issues_opened'] += 1
                if issue['state'] == 'closed':
                    self.contributors[contributor]['issues_closed'] += 1

    def _get_comments_stats(self, contributor: str, since_date: datetime) -> None:
        """Получение статистики комментариев"""
        # Комментарии к issues и PR
        comments_url = f"{self.base_url}/repos/{self.repo}/issues/comments"
        params = {
            'since': since_date.isoformat()
        }

        comments = self._make_request(comments_url, params)

        for comment in comments:
            if comment['user']['login'] == contributor:
                self.contributors[contributor]['comments'] += 1
                # Определяем тип комментария (PR или Issue)
                issue_url = comment.get('issue_url', '')
                if '/pulls/' in issue_url:
                    self.contributors[contributor]['pr_comments'] += 1
                else:
                    self.contributors[contributor]['issue_comments'] += 1

    def get_top_contributors(self, top_n: int = 5) -> List[Tuple[str, Dict]]:
        """Получение топ N контрибьюторов по активности"""
        sorted_contributors = sorted(
            self.contributors.items(),
            key=lambda x: x[1]['activity_score'],
            reverse=True
        )
        return sorted_contributors[:top_n]

    def print_report(self, top_contributors: List[Tuple[str, Dict]], min_commits: int):
        """Вывод отчета в консоль"""
        print("\n" + "="*80)
        print(f"☑ ТОП-5 АКТИВНЫХ КОНТРИБЬЮТОРОВ В {self.repo.upper()}")
        print("="*80)

        for i, (login, stats) in enumerate(top_contributors, 1):
            print(f"\n{i}. @{login}")
            print(f"   ├─ 📝 Коммиты: {stats['commits']}")
            print(f"   ├─ 💻 Изменения кода: +{stats['additions']} / -{stats['deletions']}
строк")
            print(f"   ├─ 🔀 Pull Requests: открыто {stats['prs_opened']} / закрыто
{stats['prs_closed']}")
            print(f"   ├─ 🐛 Issues: открыто {stats['issues_opened']} / закрыто
{stats['issues_closed']}")
            print(f"   ├─ 💬 Комментарии: {stats['comments']} (PR: {stats['pr_comments']},
Issues: {stats['issue_comments']})")
            print(f"   └─ ☆ Активность: {stats['activity_score']:.1f} баллов")

        print("\n" + "="*80)

    def visualize_activity(self, top_contributors: List[Tuple[str, Dict]], filename: str
= None):
        """Визуализация активности контрибьюторов"""
        if not top_contributors:
            print("Нет данных для визуализации")
```

```
            return

        if filename is None:
            filename = f"{self.repo_name}_contributors.png"

        # Подготовка данных
        logins = [f"@{login}" for login, _ in top_contributors]
        commits = [stats['commits'] for _, stats in top_contributors]
        prs = [stats['prs_opened'] for _, stats in top_contributors]
        issues = [stats['issues_opened'] for _, stats in top_contributors]
        comments = [stats['comments'] for _, stats in top_contributors]

        # Создание фигуры с несколькими подграфиками
        fig, axes = plt.subplots(2, 2, figsize=(14, 10))
        fig.suptitle(f'Активность контрибьюторов в репозитории {self.repo}',
                     fontsize=16, fontweight='bold')

        # График 1: Коммиты
        colors = plt.cm.viridis(np.linspace(0, 0.9, len(logins)))
        axes[0, 0].bar(logins, commits, color=colors, alpha=0.8)
        axes[0, 0].set_title('Количество коммитов', fontsize=12, fontweight='bold')
        axes[0, 0].set_ylabel('Коммиты')
        axes[0, 0].tick_params(axis='x', rotation=45)

        # Добавляем значения на столбцы
        for i, v in enumerate(commits):
            axes[0, 0].text(i, v + max(commits)*0.01, str(v), ha='center', fontsize=9)

        # График 2: Pull Requests и Issues (сгруппированные)
        x = np.arange(len(logins))
        width = 0.35
        axes[0, 1].bar(x - width/2, prs, width, label='Pull Requests', color='#2ecc71',
alpha=0.8)
        axes[0,  1].bar(x  +  width/2,  issues,  width,  label='Issues',  color='#e74c3c',
alpha=0.8)
        axes[0, 1].set_title('Pull Requests и Issues', fontsize=12, fontweight='bold')
        axes[0, 1].set_ylabel('Количество')
        axes[0, 1].set_xticks(x)
        axes[0, 1].set_xticklabels(logins, rotation=45)
        axes[0, 1].legend()

        # График 3: Комментарии
        axes[1, 0].bar(logins, comments, color='#3498db', alpha=0.8)
        axes[1, 0].set_title('Количество комментариев', fontsize=12, fontweight='bold')
        axes[1, 0].set_ylabel('Комментарии')
        axes[1, 0].tick_params(axis='x', rotation=45)

        # Добавляем значения на столбцы
        for i, v in enumerate(comments):
            axes[1, 0].text(i, v + max(comments)*0.01, str(v), ha='center', fontsize=9)

        # График 4: Общая активность (радарная диаграмма для топ-3)
        if len(top_contributors) >= 3:
            metrics = ['Коммиты', 'PR', 'Issues', 'Комментарии', 'Изменения\n(норм.)']
            top3_data = []
            for _, stats in top_contributors[:3]:
                normalized = [
                    stats['commits'] / max(commits),
                    stats['prs_opened'] / max(prs) if max(prs) > 0 else 0,
                    stats['issues_opened'] / max(issues) if max(issues) > 0 else 0,
                    stats['comments'] / max(comments) if max(comments) > 0 else 0,
                    min((stats['additions'] + stats['deletions']) / 1000, 1)  # Нормируем
до 1
                ]
                top3_data.append(normalized)

            # Радарная диаграмма
            angles = np.linspace(0, 2 * np.pi, len(metrics), endpoint=False).tolist()
            angles += angles[:1]
```

```
            ax = plt.subplot(2, 2, 4, projection='polar')
            colors_radar = ['#e74c3c', '#3498db', '#2ecc71']

            for i, data in enumerate(top3_data):
                data += data[:1]
                ax.plot(angles,            data,            'o-',            linewidth=2,
label=f'@{top_contributors[i][0]}', color=colors_radar[i])
                ax.fill(angles, data, alpha=0.1, color=colors_radar[i])

            ax.set_xticks(angles[:-1])
            ax.set_xticklabels(metrics)
            ax.set_title('Сравнительная активность (нормированные показатели)',
                         fontsize=12, fontweight='bold', pad=20)
            ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.0))
        else:
            axes[1, 1].text(0.5, 0.5, 'Недостаточно данных\nдля радарной диаграммы',
                            ha='center', va='center', fontsize=12)
            axes[1,      1].set_title('Сравнительная      активность',      fontsize=12,
fontweight='bold')

        plt.tight_layout()
        plt.savefig(filename, dpi=150, bbox_inches='tight')
        print(f"\n📊 Графики активности сохранены в '{filename}'")
        plt.show()

    def generate_html_report(self, top_contributors: List[Tuple[str, Dict]], filename: str
= None):
        """Генерация HTML отчета"""
        if filename is None:
            filename = f"{self.repo_name}_report.html"

        html_content = f"""
        <!DOCTYPE html>
        <html>
        <head>
            <meta charset="UTF-8">
            <title>GitHub Activity Report - {self.repo}</title>
            <style>
                body {{
                    font-family: Arial, sans-serif;
                    margin: 40px;
                    background-color: #f5f5f5;
                }}
                .container {{
                    max-width: 1200px;
                    margin: 0 auto;
                    background-color: white;
                    padding: 20px;
                    border-radius: 10px;
                    box-shadow: 0 0 10px rgba(0,0,0,0.1);
                }}
                h1 {{
                    color: #24292e;
                    border-bottom: 3px solid #0366d6;
                    padding-bottom: 10px;
                }}
                .contributor {{
                    background-color: #f6f8fa;
                    border-left: 4px solid #0366d6;
                    margin: 20px 0;
                    padding: 15px;
                    border-radius: 5px;
                }}
                .contributor h3 {{
                    margin-top: 0;
                    color: #0366d6;
                }}
                .stats {{
```

```
                    display: grid;
                    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
                    gap: 10px;
                    margin-top: 10px;
                }}
                .stat {{
                    background-color: white;
                    padding: 8px;
                    border-radius: 3px;
                    box-shadow: 0 1px 2px rgba(0,0,0,0.05);
                }}
                .stat-value {{
                    font-size: 20px;
                    font-weight: bold;
                    color: #24292e;
                }}
                .stat-label {{
                    font-size: 12px;
                    color: #586069;
                }}
                .badge {{
                    display: inline-block;
                    padding: 2px 6px;
                    border-radius: 12px;
                    font-size: 12px;
                    font-weight: bold;
                }}
                .badge-commits {{ background-color: #28a745; color: white; }}
                .badge-prs {{ background-color: #0366d6; color: white; }}
                .badge-issues {{ background-color: #e36209; color: white; }}
                .badge-comments {{ background-color: #6f42c1; color: white; }}
            </style>
        </head>
        <body>
            <div class="container">
                <h1>📊 GitHub Contributor Activity Report</h1>
                <p><strong>Репозиторий:</strong> {self.repo}</p>
                <p><strong>Дата    отчета:</strong>    {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}</p>

                <h2>🏆 Топ-5 активных контрибьюторов</h2>
        """

        for rank, (login, stats) in enumerate(top_contributors, 1):
            html_content += f"""
                <div class="contributor">
                    <h3>{rank}. @{login}</h3>
                    <div class="stats">
                        <div class="stat">
                            <div class="stat-value">{stats['commits']}</div>
                            <div class="stat-label">📝 Коммиты</div>
                        </div>
                        <div class="stat">
                            <div              class="stat-value">+{stats['additions']}/-
{stats['deletions']}</div>
                            <div class="stat-label">💻 Изменения кода</div>
                        </div>
                        <div class="stat">
                            <div                                          class="stat-
value">{stats['prs_opened']}/{stats['prs_closed']}</div>
                            <div class="stat-label">🔀 PR (открыто/закрыто)</div>
                        </div>
                        <div class="stat">
                            <div                                          class="stat-
value">{stats['issues_opened']}/{stats['issues_closed']}</div>
                            <div class="stat-label">🐛 Issues (открыто/закрыто)</div>
                        </div>
                        <div class="stat">
```

```
                            <div class="stat-value">{stats['comments']}</div>
                            <div class="stat-label">💬 Комментарии</div>
                        </div>
                        <div class="stat">
                            <div class="stat-value">{stats['activity_score']:.1f}</div>
                            <div class="stat-label">⭐ Балл активности</div>
                        </div>
                    </div>
                    <div>
                        <span            class="badge            badge-commits">Коммиты:
{stats['commits']}</span>
                        <span class="badge badge-prs">PR: {stats['prs_opened']}</span>
                        <span             class="badge             badge-issues">Issues:
{stats['issues_opened']}</span>
                        <span          class="badge          badge-comments">Комментарии:
{stats['comments']}
                    </div>
                </div>
            """

        html_content += """
            </div>
        </body>
        </html>
        """

        with open(filename, 'w', encoding='utf-8') as f:
            f.write(html_content)

        print(f"📄 HTML отчет сохранен в '{filename}'")


def main():
    """Основная функция"""
    print("="*80)
    print("🐙 GitHub Contributor Activity Analyzer")
    print("Анализ активности контрибьюторов в GitHub репозитории")
    print("="*80)

    # Ввод данных от пользователя
    repo = input("\nВведите репозиторий (owner/repo): ").strip()
    if not repo or '/' not in repo:
        print("Ошибка: неверный формат репозитория. Используйте формат owner/repo")
        sys.exit(1)

    print("\nВыберите период анализа:")
    print("1. Неделя (7 дней)")
    print("2. Месяц (30 дней)")
    print("3. Год (365 дней)")
    period_choice = input("Ваш выбор (1-3): ").strip()

    period_map = {
        '1': 'week',
        '2': 'month',
        '3': 'year'
    }

    period_key = period_map.get(period_choice, 'month')
    days = {'week': 7, 'month': 30, 'year': 365}[period_key]
    since_date = datetime.now() - timedelta(days=days)

    min_commits_input = input("\nМинимальное количество коммитов для попадания в рейтинг
(по умолчанию 0): ").strip()
    min_commits = int(min_commits_input) if min_commits_input else 0

    # Опционально: GitHub токен
    token  =  input("\nGitHub  токен  (опционально,  для  увеличения  лимита  запросов):
").strip()
    if not token:
```

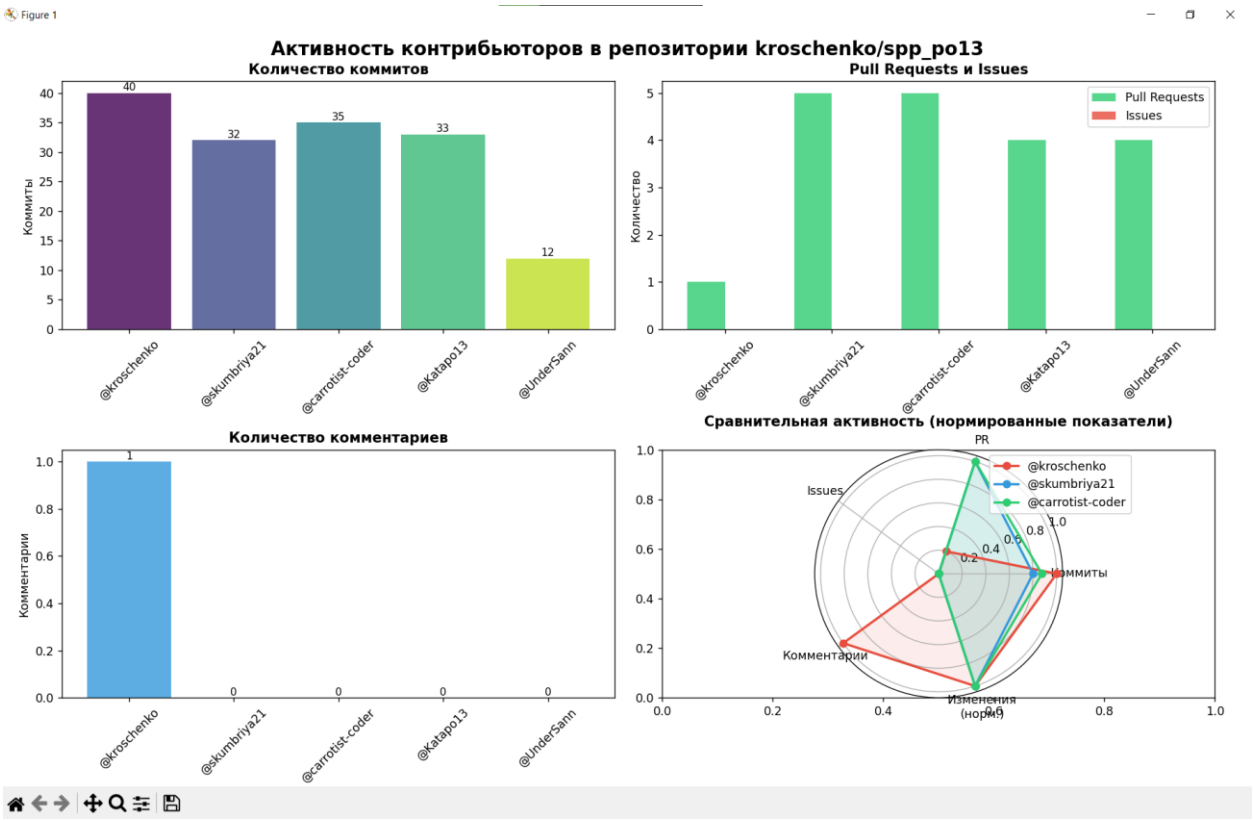
```
        token = None

    print("\n" + "="*80)
    print("🚀 Начинаем анализ...")

    try:
        # Создаем анализатор
        analyzer = GitHubContributorAnalyzer(repo, token)

        # Получаем статистику
        contributors = analyzer.get_contributors_stats(since_date, min_commits)

        if not contributors:
            print("\n❌ Не найдено контрибьюторов, удовлетворяющих критериям")
            return

        # Получаем топ-5
        top_contributors = analyzer.get_top_contributors(5)

        # Выводим отчет
        analyzer.print_report(top_contributors, min_commits)

        # Визуализируем результаты
        analyzer.visualize_activity(top_contributors)

        # Генерируем HTML отчет
        analyzer.generate_html_report(top_contributors)

        print("\n✨ Анализ успешно завершен!")

    except Exception as e:
        print(f"\n❌ Произошла ошибка: {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)


if __name__ == "__main__":
    main()
```

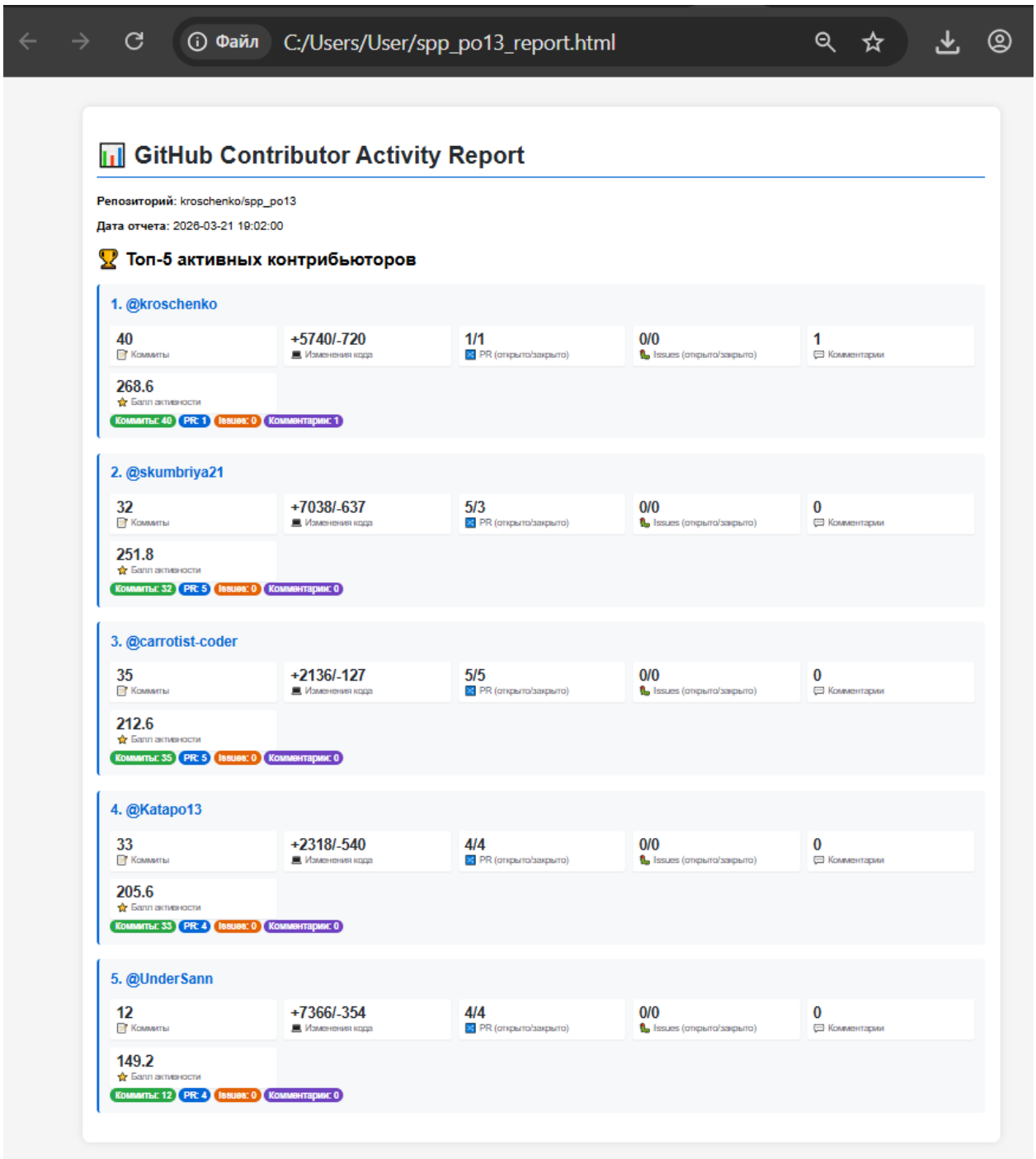
```
================================================================================
  GitHub Contributor Activity Analyzer
Анализ активности контрибьюторов в GitHub репозитории
================================================================================

Введите репозиторий (owner/repo): Katapo13/Guide-robot-Keenon

Выберите период анализа:
1. Неделя (7 дней)
2. Месяц (30 дней)
3. Год (365 дней)
Ваш выбор (1-3): 2

Минимальное количество коммитов для попадания в рейтинг (по умолчанию 0): 0

GitHub токен (опционально, для увеличения лимита запросов): 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

================================================================================
  Начинаем анализ...
  Используется GitHub токен для аутентификации

  Анализ контрибьюторов репозитория Katapo13/Guide-robot-Keenon
  Период анализа: с 2026-02-19 по 2026-03-21
  Минимальное количество коммитов: 0

1. Получение списка контрибьюторов...
   Найдено 2 контрибьюторов (после фильтрации)

2. Сбор детальной статистики...
   Сбор данных завершен


================================================================================
  ТОП-5 АКТИВНЫХ КОНТРИБЬЮТОРОВ В KATAPO13/GUIDE-ROBOT-KEENON
================================================================================

1. @Nihilclsd
   ├─  Коммиты: 8
   ├─  Изменения кода: +51 / -13 строк
   ├─  Pull Requests: открыто 0 / закрыто 0
   ├─  Issues: открыто 0 / закрыто 0
   ├─  Комментарии: 0 (PR: 0, Issues: 0)
   └─  Активность: 40.6 баллов

2. @Katapo13
   ├─  Коммиты: 1
   ├─  Изменения кода: +0 / -0 строк
   ├─  Pull Requests: открыто 0 / закрыто 0
   ├─  Issues: открыто 0 / закрыто 0
   ├─  Комментарии: 0 (PR: 0, Issues: 0)
   └─  Активность: 5.0 баллов

================================================================================

  Графики активности сохранены в 'Guide-robot-Keenon_contributors.png'
```

```
================================================================================
□  GitHub Contributor Activity Analyzer
Анализ активности контрибьюторов в GitHub репозитории
================================================================================

Введите репозиторий (owner/repo): kroschenko/spp_po13

Выберите период анализа:
1. Неделя (7 дней)
2. Месяц (30 дней)
3. Год (365 дней)
Ваш выбор (1-3): 3

Минимальное количество коммитов для попадания в рейтинг (по умолчанию 0): 5

GitHub токен (опционально, для увеличения лимита запросов): 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

================================================================================
□  Начинаем анализ...
□ Используется GitHub токен для аутентификации

□  Анализ контрибьюторов репозитория kroschenko/spp_po13
□  Период анализа: с 2025-03-21 по 2026-03-21
□  Минимальное количество коммитов: 5

1. Получение списка контрибьюторов...
   Найдено 14 контрибьюторов (после фильтрации)

2. Сбор детальной статистики...
   Обработано 20/56 задач...
   Обработано 40/56 задач...
   Сбор данных завершен


================================================================================
□  ТОП-5 АКТИВНЫХ КОНТРИБЬЮТОРОВ В KROSCHENKO/SPP_PO13
================================================================================

1. @kroschenko
   ├─ □  Коммиты: 40
   ├─ □  Изменения кода: +5740 / -720 строк
   ├─ □  Pull Requests: открыто 1 / закрыто 1
   ├─ □  Issues: открыто 0 / закрыто 0
   ├─ □  Комментарии: 1 (PR: 0, Issues: 1)
   └─ □ Активность: 268.6 баллов

2. @skumbriya21
   ├─ □  Коммиты: 32
   ├─ □  Изменения кода: +7038 / -637 строк
   ├─ □  Pull Requests: открыто 5 / закрыто 3
   ├─ □  Issues: открыто 0 / закрыто 0
   ├─ □  Комментарии: 0 (PR: 0, Issues: 0)
   └─ □ Активность: 251.8 баллов
```

```
3. @carrotist-coder
   ├─ □ Коммиты: 35
   ├─ □ Изменения кода: +2136 / -127 строк
   ├─ □ Pull Requests: открыто 5 / закрыто 5
   ├─ □ Issues: открыто 0 / закрыто 0
   ├─ □ Комментарии: 0 (PR: 0, Issues: 0)
   └─ □ Активность: 212.6 баллов

4. @Katapo13
   ├─ □ Коммиты: 33
   ├─ □ Изменения кода: +2318 / -540 строк
   ├─ □ Pull Requests: открыто 4 / закрыто 4
   ├─ □ Issues: открыто 0 / закрыто 0
   ├─ □ Комментарии: 0 (PR: 0, Issues: 0)
   └─ □ Активность: 205.6 баллов

5. @UnderSann
   ├─ □ Коммиты: 12
   ├─ □ Изменения кода: +7366 / -354 строк
   ├─ □ Pull Requests: открыто 4 / закрыто 4
   ├─ □ Issues: открыто 0 / закрыто 0
   ├─ □ Комментарии: 0 (PR: 0, Issues: 0)
   └─ □ Активность: 149.2 баллов

================================================================================

□ Графики активности сохранены в 'spp_po13_contributors.png'
```

```
□ Графики активности сохранены в 'spp_po13_contributors.png'
□ HTML отчет сохранен в 'spp_po13_report.html'

□ Анализ успешно завершен!
```





Вывод: были отработаны навыки работы с Github API, приобретены практические навыки написания программ для работы с REST API.